

Programowanie współbieżne w C++

Lista zadań nr 12

Na pracownię 28 maja 2026 (i być może
późniejszą)

Poniższe zadania 1 i 2 dotyczą kolejki wielowątkowej z rozdziału 7.2.6 książki C++ Concurrency in Action, 2e. Zadanie 1 dotyczy implementacji omawianej na wykładzie. Zadanie 2 – implementacji, w której szybsze wątki pomagają wolniejszym czyniąc tę implementację prawdziwie lock-free (listingi 7.21 i 7.22).

Zadanie 1 (7pktów). Kolejka wielowątkowa bez blokad. Zwalnianie pamięci przy pomocy liczników referencji. (wariant bez wątków szybszych pomagających wolniejszym)

Uruchom i przetestuj kolejkę wielowątkową z rozdziału 7.2.6 książki C++ Concurrency in Action, 2e. Upewnij się, że rozumiesz jak działa mechanizm zwalniania pamięci przy pomocy liczników referencji. W szczególności odpowiedz na pytania:

- a) Dlaczego węzłów w tej strukturze nie można zwalniać bezpośrednio w funkcji `pop()`, bez dodatkowych zabiegów?
- b) Do czego służą zmienne `internal_count`, `external_count` oraz `external_counters`? Jakie są początkowe wartości tych zmiennych i jak mogą się one zmieniać podczas wykonania programu?
- c) Jaka jest rola funkcji `increase_external_count()`, `free_external_counter()` oraz `release_ref()`?
- d) Dlaczego `release_ref()` używa `compare_exchange_strong()` zamiast `fetch_sub()` przy zmniejszaniu licznika?
- e) W jakim przypadku może dojść do podwójnego `delete`, jeśli zignorujemy `external_counters`?
- f) Co się stanie, jeśli zapomnimy wywołać `free_external_counter()` po udanym `compare_exchange` na `head` lub `tail`?
- g) Dlaczego `compare_exchange_strong()` w `release_ref()` i `free_external_counter()` używa `memory_order_acquire` / `memory_order_relaxed`?
- h) Dlaczego potrzebujemy `memory_order_acquire` w `increase_external_count()`?

- i) Dlaczego `pop()` ładuje `head` z `memory_order_relaxed`, ale potem wykonuje `compare_exchange_strong()`?
- j) Czy można by zastosować `memory_order_seq_cst` wszędzie dla bezpieczeństwa? Dlaczego tego nie robimy?
- k) Co może pójść nie tak, jeśli dwa wątki jednocześnie wywołają `push()` i oba wykonają `compare_exchange_strong()` na tym samym ogonie `tail` bez zwiększenia `external_count`?
- l) Dlaczego, mimo braku użycia muteksów i jawnego zastosowania innych blokad, implementację tę nie możemy uznać że w pełni lock-free?

Zadanie 2 (7pktów). Kolejka z pomocnymi wątkami

Uruchom i przetestuj kolejkę wielowątkową, w której w funkcji `push()` szybsze wątki pomagają wolniejszemu. Upewnij się, że rozumiesz jej zasadę działania. Odpowiedz na pytania:

- a) Jakie zmiany zaszły w kodzie, w porównaniu do kolejki z poprzedniego zadania?
- b) Dlaczego zmienna `next` w węźle musi być typu `std::atomic<counted_node_ptr>` i czemu jej wczytywanie w `pop()` musi być atomowe?
- c) Jak działa funkcja `set_new_tail()`?
- d) Czy ta implementacja jest w pełni lock-free?

Zadanie 3 (5pktów). Epoch-Based Reclamation w praktyce.

Wybierz dowolne zadanie z poprzednich list, w którym problemem była bezpieczna dealokacja pamięci. Zaimplementuj w nim mechanizm EBR.

Przed rozpoczęciem rozwiązywania Zadania 4, przeczytaj notatkę dotyczącą zastosowania Interval-Based Reclamation, zamieszczoną na stronie wykładu.

Zadanie 4 (7pktów). Interval-Based Reclamation w praktyce.

Wybierz dowolne zadanie z poprzednich list, w którym problemem była bezpieczna dealokacja pamięci. Zaimplementuj w nim mechanizm IBR.